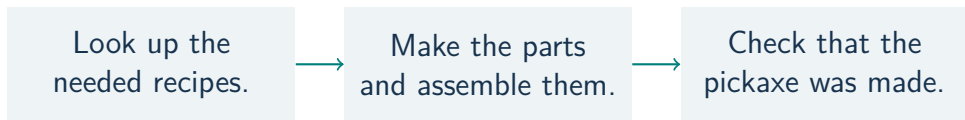


How can a model solve an unfamiliar task?

We want a language model to discover useful steps, use tools, and check whether the job is finished.

A simple test: make a pickaxe in a crafting game.



The game checks actual success, not whether the answer sounds convincing.

The larger idea is a **recursive language model (RLM)**: a model that can use code and ask helper models to solve smaller parts.

Simplified example. Our recent crafting experiments use one model with tools; they do not yet test recursive helper calls.

We tried learning from success and failure.

In **reinforcement learning (RL)**, the model tries answers or actions, receives a score, and learns from the feedback.

We saw some gains, but not reliable progress in planning.

Some models answered or selected information better. The gains often weakened on new examples, or did not clearly beat more learning from worked examples.

Why is rewarding a finished task tricky?

The model asks for a recipe, makes a wrong part, corrects it, and finishes the pickaxe. A reward for finishing does not say which earlier steps helped.

Next RL question: How can feedback teach useful steps, not just reward the final result?

This illustrates one possible challenge. It has not been proven to explain all our RL results.

Can examples teach the model how to find the steps?

We trained the model to imitate actions from worked examples. The goal is to make a pickaxe.

Teacher knows the recipes

It plans the parts first, then demonstrates recipe lookups and crafting.

Both examples show lookups. Only the second uses their answers to choose the route.

Successes out of 32 attempts: **discovery examples: 15, then 10;**
known-recipe examples: 5, then 3.

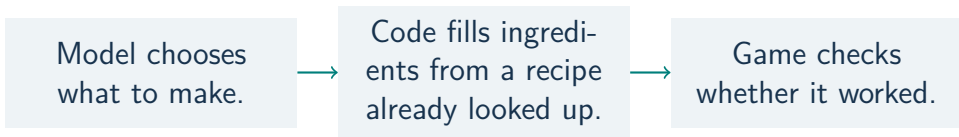
Simplified examples; both training packages differ in other ways. Two training repeats in the same game, not independent datasets.

Teacher discovers the recipes

It looks up the pickaxe, finds its parts, then chooses the steps.

Who should supply the exact ingredients?

The model chooses the next step and amount: “Make one pickaxe.”



Example: The model asks to make one pickaxe. Code supplies the ingredient list from the recipe the model already found.

No extra model training. On the first test of additional goals, success was **7 of 16 without code** and **10 of 16 with code**.

This changes how the command is carried out, not the model's choice of goal. Early result in the same game; broad transfer is not established.

What are we testing now, and what comes next?

Running now: On new crafting goals and recipes, compare both teaching methods with and without code assistance.

This tests two questions: Does code help on new goals? Does the discovery advantage remain when code fills in the ingredients?

Proposed RL follow-up: Test reward learning with and without code assistance. We have not fixed the design yet.

Longer term: Learn when to ask a helper, or break a difficult part into smaller steps.

For discussion

What other multi-step task would make this useful?

The broader crafting comparison is underway. RL and learned helper use are future research, not current results.